Universidad de Antioquia
Departamento de Electrónica y Telecomunicaciones
2598521 - Informatica II

Implementación de Tetris mediante manipulación de bits en C++

Informe de Análisis y Diseño Inicial

Rigoberto Berrio Berrio
Estudiante de Ingeniería Electrónica
Universidad de Antioquia
rigoberto.berrio1@udea.edu.co

Felipe Herrera Lopera
Estudiante de Ingeniería Electrónica
Universidad de Antioquia
felipe.herreral@udea.edu.co

13 de marzo de 2026

1. Análisis del problema

El desafío técnico que tenemos entre manos con este proyecto de Tetris Bitwise no es solo recrear un juego clásico, sino repensar por completo cómo gestionamos la información en memoria. Como estudiantes, estamos acostumbrados a usar matrices de enteros o incluso objetos complejos para representar un tablero, pero aquí la restricción nos obliga a ir un paso más allá y pasar de bytes a bits.

Con esta implementación, el objetivo es demostrar que podemos dominar conceptos básicos de Informática II como la aritmética de punteros, la gestión de memoria dinámica y, sobre todo, la utilidad de operadores bitwise para que el motor del juego funcione de la manera más eficiente posible.

Uno de los aspectos importantes en el diseño del esquema de datos mediante el cual se representa el tablero es que el tablero no está concebido como una simple cuadrícula visual sino como un buffer de información cuyas anchuras deben ser múltiplos de 8; no es una decisión arbitraria, sino que de este modo permite convertir cada fila en un grupo de bytes (unsigned char) y permite un recorrido de la memoria lineal y eficiente.

Aquí es donde el reto se pone interesante, porque cada movimiento de una pieza o cada rotación de las piezas y sus diferentes orientaciones no se resuelve cambiando índices en una matriz, sino desplazando bits («, ») y aplicando máscaras lógicas (&,—); para saber si hay una colisión o si la pieza ya debe quedar fija en el tablero.

Otro punto crítico que hemos analizado es la limpieza de las filas y el control del estado del juego. No podemos usar librerías externas ni la STL, lo que significa que nos olvidamos de std::vector o std::string; todo, desde el buffer de la pantalla hasta la estructura de las piezas, depende de nuestra capacidad para gestionar punteros crudos y memoria dinámica con new y delete. m

El éxito del motor dependerá de qué tan bien logremos que estos módulos se comuniquen para detectar un Game Over o eliminar una fila llena sin que el programa colapse o deje fugas de memoria, manteniendo siempre la lógica modular que exige el framework de Qt.

2. Diseño del Sistema

El diseño se fundamenta en una arquitectura modular que separa la lógica basada en bits de la interfaz. Para ello, se emplean punteros y memoria dinámica en lugar de librerías automáticas como la STL. Con el fin de lograr un motor eficiente, el sistema se divide en tres componentes principales: un gestor de tablero, encargado de manejar la memoria cruda; un módulo de piezas, que contiene las formas predefinidas representadas en 16 bits; y un motor de colisiones, que utiliza operaciones lógicas para validar cada movimiento del jugador.

La comunicación entre estos componentes se realiza de manera directa. La interfaz se encarga de capturar las teclas ingresadas por el usuario y solicita al motor que desplace o rote los bits correspondientes a la pieza actual. Si el motor determina, mediante la aplicación de máscaras lógicas, que no existe colisión, el tablero se actualiza y se redibuja en la consola utilizando caracteres simples. De esta manera, se mantiene un control completo sobre el ciclo de vida de los datos y se evita la aparición de fugas de memoria.

2.1. Componentes Principales

- **Módulo BitCore (Tablero):** Representa la unidad fundamental de almacenamiento. Utiliza un puntero dinámico de tipo unsigned char* para gestionar el tablero como una secuencia de bytes, donde cada bit encendido indica un bloque ocupado.

- **Gestor de Tetriminos:** Define las 7 piezas clásicas y sus estados de rotación. Cada pieza se almacena en estructuras de 16 bits para permitir comparaciones rápidas de colisión mediante el operador AND (&).

- **Motor de Colisiones y Gravedad:** Administra la lógica de movimiento. Valida si un desplazamiento es posible comparando las máscaras de la pieza actual con los bits ocupados en el tablero.

- **Limpiador de Filas:** Implementa la detección de filas llenas (todos los bits en 1) y gestiona la caída de las filas superiores mediante el desplazamiento de datos o reasignación de punteros.

- **Interfaz de Consola:** Controlador encargado de la entrada de usuario y la representación visual mediante caracteres, recorriendo el buffer de bits para imprimir el estado actual.

2.2. Relaciones Lógicas

- **Tablero:** Agrega y contiene el estado de los tetriminos una vez fijados.
- **Motor de Juego:** Gestiona el ciclo de vida del tablero y solicita datos al gestor de términos.
- **Interfaz:** Usa el estado del tablero para renderizar y envía comandos al motor de Juego.

3. Posibles Desafíos en la Implementación

- **Gestión de fronteras y colisiones laterales:** A medida que se avance en la codificación, se prevé que el mayor reto será la gestión de fronteras y colisiones laterales. Al no trabajar con una matriz de coordenadas (x, y) tradicional, sino con bits desplazados, detectar que una pieza ha tocado el borde del tablero requiere un uso muy preciso de máscaras. Si se falla en un solo bit de desplazamiento, la pieza podría .ªtravesar"las paredes o aparecer en el lado opuesto del tablero, lo que obligará a ser extremadamente cuidadosos con la aritmética de punteros.
- **Limpieza de filas dinámicas:** El requisito de que las filas superiores bajen cuando una se llena suena sencillo en papel, pero al no usar la STL, se tendrá que decidir entre mover físicamente todos los bits hacia abajo o realizar un intercambio de las direcciones de memoria de los punteros de cada fila. Esta última opción es más eficiente pero requiere un manejo impecable de la memoria dinámica para no perder la referencia de ningún byt.

4. Diagrama preliminar de TETRIX

El proceso inicia con la reserva de memoria dinámica para el tablero basada en un ancho múltiplo de ocho, estableciendo un ciclo de juego por turnos donde cada movimiento o rotación se valida mediante una proyección bit a bit. El motor utiliza la operación lógica AND para detectar colisiones de forma instantánea y el operador OR para fijar las piezas una vez que agotan su descenso; finalmente, el flujo analiza la saturación de bits por fila para ejecutar la limpieza mediante la reasignación de punteros o desplazamientos, garantizando un control estricto de la memoria y el estado de Game Over sin depender de librerías externas.

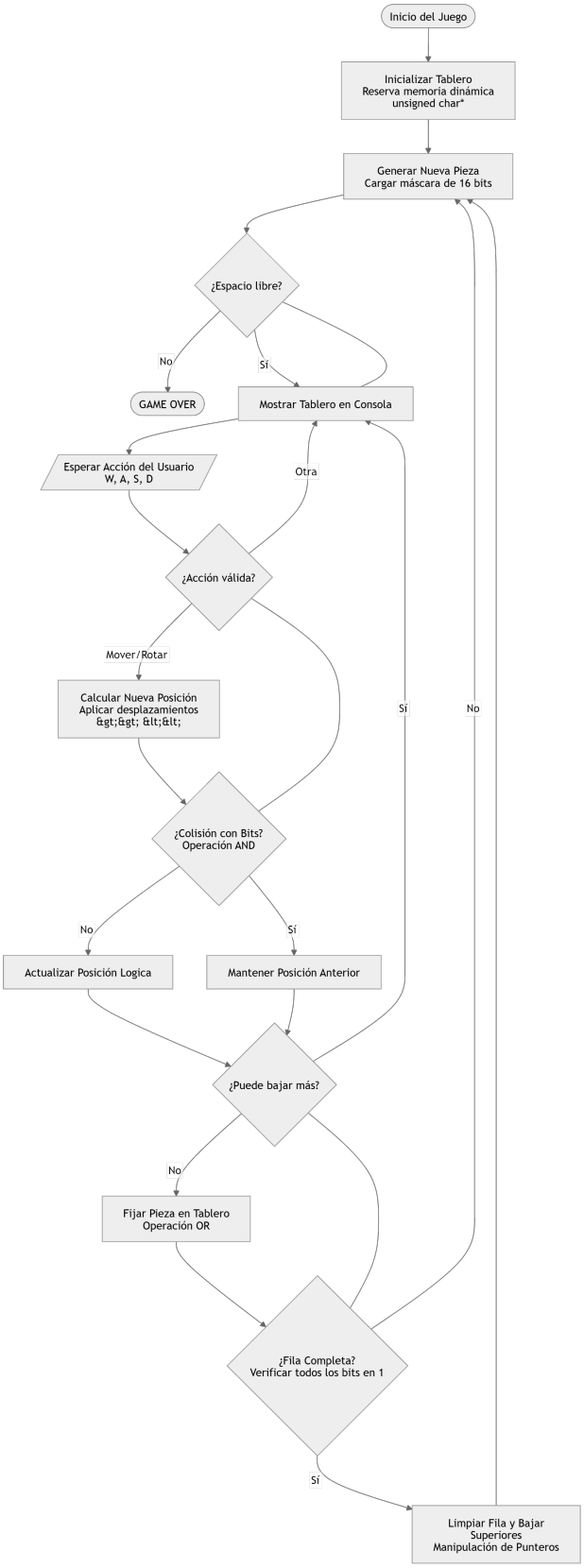

Figura 1: Diagrama del Sistema de Juego

5. Conclusiones

En esta fase preliminar, se ha establecido una base sólida que traduce la mecánica del Tetris a un entorno de manipulación de bits puro. El análisis detallado permitió identificar que la mayor complejidad reside en el manejo de un ancho de tablero dinámico bajo la restricción de múltiplos de 8, lo cual se resolverá mediante aritmética de punteros y máscaras de bits.

El diseño propuesto asegura una organización modular que facilita la escalabilidad y depuración, cumpliendo estrictamente con la prohibición de librerías externas o contenedores STL. Este avance garantiza una estructura coherente, lista para la etapa de implementación en Qt, donde se priorizará la correcta gestión de los ciclos de vida de la memoria dinámica.